

ClassLedger

Registre pédagogique sur blockchain privée

Enseignant responsable	Nouredine Tamani
Plateforme	Hyperledger Besu sur Microsoft Azure
Application	Python / FastAPI
Contrat	Solidity (EVM)
État de l'art arrêté au	5 septembre 2026
Dernière validation	8 septembre 2026 — contrat V4

Sommaire

Résumé

1. Problème à résoudre

2. Étude des solutions et choix de plateforme

3. Architecture retenue

4. Modèle on-chain et règles d'accès

5. Réseau et consensus

6. Application et sécurité

7. Démarche de réalisation

8. Validation

9. Déploiement Azure

10. Limites et portée des résultats

Conclusion

Références

Résumé

ClassLedger applique la blockchain à un cas volontairement limité : le partage des ressources d'une classe. La description et l'organisation sont publiques ; les cours et travaux pratiques sont réservés aux inscrits ; les examens et leurs corrections ne sont ouverts que vingt-quatre heures après l'épreuve ; enfin, chaque étudiant ne peut consulter que sa propre note.

La difficulté ne consiste pas à afficher ces informations dans une application web, mais à faire porter les règles par un registre distribué. La solution repose donc sur un smart contract Solidity exécuté par quatre validateurs Hyperledger Besu. Le consensus QBFT permet de finaliser les blocs tant que trois validateurs sur quatre participent. FastAPI joue le rôle de passerelle entre les utilisateurs et la chaîne. Les fichiers restent sur disque, tandis que leur empreinte SHA-256 et leurs métadonnées sont ancrées dans le contrat.

La réalisation a été conduite par incréments : formalisation des règles, comparaison des plateformes, conception du contrat, mise en réseau, ajout de la passerelle, sécurisation, puis validation. La campagne finale comprend 23 tests automatisés, un parcours de bout en bout et un essai de quorum réel. Elle confirme la réplication des blocs, l'arrêt du consensus sous le seuil requis, le rattrapage des nœuds et la détection d'un fichier altéré. Le système a également été déployé sur une VM Azure avec HTTPS et des ports blockchain non exposés.

1. Problème à résoudre

Le sujet impose quatre règles fonctionnelles et quatre propriétés propres à une blockchain. Leur traduction en exigences vérifiables a constitué le point de départ du travail.

Réf.	Exigence	Critère observable
R1	Description et organisation publiques	lecture possible sans compte
R2	Cours et TP réservés aux inscrits	refus avant inscription, accès après approbation
R3	Examens et corrections ouverts après 24 h	refus avant l'échéance, accès ensuite
R4	Note personnelle uniquement	un étudiant ne lit jamais la note d'un autre
B1	Réplication et auditabilité	les quatre nœuds possèdent le même bloc ; les informations publiques sont lisibles
B2	Immutabilité des publications	documents, informations de classe et notes sont conservés ; une révision ajoute une entrée
B3	Distribution P2P	chaque validateur possède trois pairs
B4	Consensus	3/4 finalisent ; 2/4 ne finalisent pas

Ce découpage évite un piège fréquent : reproduire une base de données classique derrière une interface qui porte seule les autorisations. Ici, une lecture privée est effectuée avec l'adresse de l'utilisateur comme `msg.sender`, et le contrat décide si elle est autorisée. L'interface ne constitue donc pas la source de vérité des quatre règles.

Le périmètre reste celui d'un prototype d'enseignement : une classe, un enseignant et des étudiants fictifs. Les fonctions d'un ENT complet — SSO, notifications, messagerie, gestion multi-classes — n'ont pas été ajoutées, car elles n'apportent rien à la démonstration du registre distribué.

2. Étude des solutions et choix de plateforme

L'étude a porté sur les offres cloud citées dans le sujet et sur deux plateformes open source adaptées aux réseaux privés.

2.1 Fournisseurs cloud

Microsoft Azure. Azure Blockchain Service a été retiré en septembre 2021 ; la documentation de migration proposait notamment un déploiement autogéré sur des machines virtuelles [2]. Azure propose encore Confidential Ledger, un registre append-only protégé par des enclaves matérielles, accessible par API REST et capable de fournir des reçus de transactions. Ce service convient à un journal d'audit, mais ne fournit pas l'environnement EVM/Solidity retenu pour exécuter les règles R2 à R4 [1].

Amazon Web Services. Amazon Managed Blockchain fournit des accès gérés à plusieurs réseaux publics ainsi qu'une offre Hyperledger Fabric pour les réseaux privés. La facturation dépend notamment des nœuds, du stockage, des requêtes et du transfert. Cette solution réduit l'administration, mais masque une partie du réseau et ajoute un coût peu justifié pour un prototype court [3].

IBM et Oracle. IBM indique que Blockchain Platform 2.5.4 n'est plus publié et oriente vers le support Hyperledger Fabric [4]. Oracle maintient une offre blockchain managée, riche en intégrations et destinée aux réseaux d'entreprise. Elle dépasse les besoins et le budget de cette application mono-classe [5].

2.2 Plateformes open source

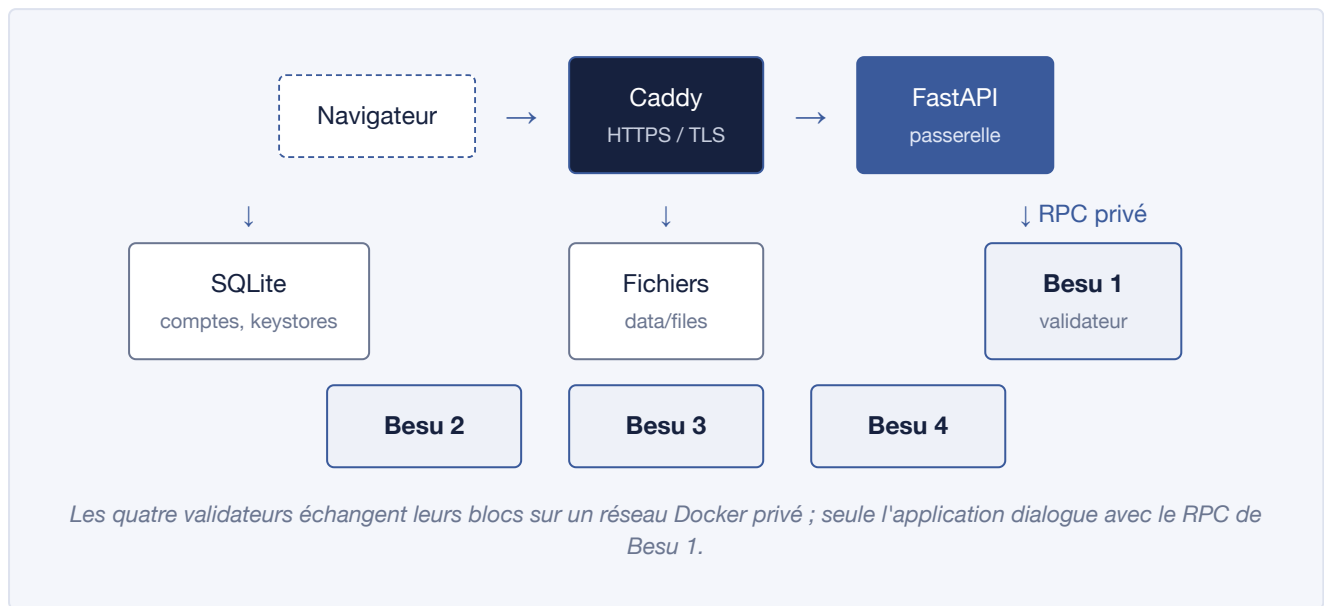
Hyperledger Fabric offre des identités, des canaux, des politiques d'endorsement et des données privées. Ces mécanismes sont pertinents dans un consortium, mais exigent davantage de composants : autorités de certification, peers, service d'ordre et configuration des canaux. Cette richesse aurait déplacé l'effort vers l'administration du réseau [8].

Hyperledger Besu est un client Ethereum open source. Il exécute des contrats Solidity, expose les API compatibles Ethereum et prend en charge QBFT pour les réseaux privés. La documentation Besu recommande QBFT pour ce contexte et précise qu'il faut quatre validateurs pour obtenir une tolérance byzantine [6]. Son modèle EVM s'intègre directement à `web3.py`.

Critère	Azure Confidential Ledger	AWS / Fabric géré	Fabric auto-hébergé	Besu auto-hébergé
Contrat Solidity	Non	Non pour Fabric	Non	Oui
Consensus visible et testable	Peu	Partiellement	Oui	Oui
Coût logiciel	Service payant	À l'usage	Gratuit	Gratuit
Complexité pour une classe	Faible, mais inadapté	Élevée	Élevée	Modérée
Intégration Python/EVM	Non	Indirecte	SDK Fabric	Directe

Le choix final est **Besu auto-hébergé sur Azure**. Besu fournit la blockchain et Azure fournit seulement l'infrastructure de démonstration. Cette séparation est importante : le projet n'affirme pas utiliser un service blockchain managé Azure qui n'existe plus. L'accès à la VM repose sur l'abonnement étudiant disponible ; le logiciel reste entièrement reproductible en local.

3. Architecture retenue



Les cinq services principaux sont isolés dans Docker Compose : quatre nœuds Besu et une application FastAPI. Sur Azure, Caddy ajoute la terminaison TLS. Aucun port RPC ou P2P n'est publié sur l'hôte ; à l'intérieur du réseau Docker privé, l'application dialogue avec le RPC de `validator1`, et les scripts de vérification interrogent les quatre nœuds pour comparer leurs copies.

Le système sépare les données selon leur nature :

- **sur la chaîne** : description, organisation, inscriptions, métadonnées, empreintes de fichiers, liens de versions et notes chiffrées ;
- **hors chaîne** : contenu des fichiers, comptes web, correspondance entre noms et adresses, keystores chiffrés et secrets applicatifs.

Le stockage hybride répond à deux contraintes contradictoires. Un fichier de plusieurs mégaoctets est mal adapté à un registre répliqué, mais sa preuve d'intégrité est courte. Le contrat conserve donc un SHA-256 de 32 octets. Lors du téléchargement, FastAPI recalcule l'empreinte ; une différence provoque un refus. La blockchain protège ainsi l'intégrité sans porter le volume du document.

4. Modèle on-chain et règles d'accès

4.1 Inscription

Le contrat distingue quatre états : `NONE` , `PENDING` , `ENROLLED` et `REJECTED` . Un compte étudiant demande son inscription pour lui-même. Seule l'adresse de l'enseignant peut ensuite approuver ou refuser cette demande. Un refus n'efface pas l'historique et autorise une nouvelle demande. Une seconde voie d'écriture, `enroll` , permet à l'enseignant d'inscrire directement une adresse connue : elle sert à initialiser la liste de classe, par exemple lors du chargement des comptes de démonstration. Les deux chemins restent tracés par des événements distincts.

Dans l'application, chaque étudiant possède une adresse Ethereum. La transaction de demande est signée avec la clé associée à cette adresse. Cette clé est toutefois conservée par la passerelle sous forme de keystore chiffré : la signature est bien celle du compte étudiant, mais le modèle reste custodial.

4.2 Documents

Chaque document est une structure `Material` comportant un type, un titre, une référence de fichier, son SHA-256, une date d'examen éventuelle, un horodatage et un lien vers une version précédente.

La fonction interne de visibilité applique une règle simple :

1. l'enseignant voit tous les documents ;
2. un non-inscrit ne voit aucun contenu privé ;
3. cours et TP sont ouverts aux inscrits ;
4. examens et corrections exigent en plus `block.timestamp >= examAt + 24 hours` .

Une correction doit référencer un examen ou une correction déjà rattachée à cet examen, et hériter de sa date. Le verrou de vingt-quatre heures reste ainsi celui de l'épreuve, même après plusieurs versions.

Aucune fonction ne modifie ni ne supprime un document publié. Le contrat V4 applique aussi cette conservation aux informations de classe et aux notes : une publication ajoute une version, sans remplacer les précédentes. `classInfoAt` permet de relire les informations publiques à un indice donné ; les lectures habituelles renvoient la dernière version. Les inscriptions restent un cycle d'état, dont chaque transition est tracée par un événement.

4.3 Notes

Le contrat associe une liste de versions à l'adresse de l'étudiant. `myGrade()` renvoie sa dernière note ; `gradeOf()` limite la lecture à l'étudiant concerné ou à l'enseignant. `gradeAt` et `gradeVersionCount` protègent l'historique avec la même règle. L'interface affiche la dernière note ; les versions antérieures restent consultables par les appels autorisés du contrat. Aucun événement ne contient la valeur de la note.

Ce contrôle ne suffit pas contre un opérateur capable d'inspecter le stockage brut d'un nœud. La note est donc chiffrée par **AES-256-GCM** avant la transaction. L'adresse de l'étudiant est utilisée comme donnée authentifiée : déplacer le ciphertext vers une autre adresse fait échouer son authentification. Les validateurs répliquent une preuve chiffrée, pas la note en clair.

5. Réseau et consensus

Le réseau utilise Besu 26.2.0, une chaîne privée d'identifiant `1337` et quatre validateurs statiques. Le genesis fixe une période de bloc de 2 secondes et un délai de tour de 4 secondes. Le prix minimal du gas est nul : aucune monnaie réelle n'est nécessaire pour signer les transactions du prototype.

QBFT est un consensus Proof of Authority à finalité immédiate. Un bloc proposé doit recevoir la signature d'au moins deux tiers des validateurs. Avec quatre nœuds, trois signatures sont nécessaires. La propriété de tolérance s'écrit :

$$f = \left\lfloor \frac{n-1}{3} \right\rfloor$$

Pour $n = 4$, le réseau tolère donc une défaillance. Si deux validateurs sont arrêtés, la sécurité est privilégiée à la disponibilité : aucun nouveau bloc n'est finalisé. Ce comportement, annoncé par la documentation Besu lorsque plus d'un tiers des validateurs ne participe plus, a été reproduit pendant les essais.

Les quatre nœuds s'appuient sur le même genesis, mais possèdent chacun leur clé et leur volume de chaîne. Ils échangent les blocs sur un réseau Docker privé. Cette topologie démontre la réplication et le consensus ; elle ne constitue toutefois pas une distribution physique puisque les conteneurs partagent le même hôte.

6. Application et sécurité

FastAPI fournit des pages rendues côté serveur avec Jinja2 [9]. Ce choix limite le nombre de composants : aucun frontend séparé ni API publique supplémentaire. La passerelle assure l'authentification web, la signature des transactions, le dépôt des fichiers et la traduction des refus du contrat en messages compréhensibles [10].

Les principales protections sont les suivantes :

- mots de passe hachés avec Argon2 ;
- clés étudiantes dans des keystores Ethereum chiffrés avec scrypt ;
- jeton CSRF lié à la session pour chaque opération d'écriture ;
- limitation des tentatives de connexion et d'inscription ;
- cookie `HttpOnly`, `SameSite=Strict`, durée de deux heures, `Secure` sur Azure ;
- contrôle de l'en-tête `Host` et en-têtes CSP, HSTS, `nosniff` et anti-frame ;
- taille des fichiers limitée à 10 Mio, noms assainis et permissions `0600` ;
- validation obligatoire de secrets indépendants au démarrage en production ;
- documentation OpenAPI désactivée en production.

Les secrets ne sont pas intégrés à l'image. Ils sont chargés depuis `.env`, exclu de Git. Azure génère des valeurs aléatoires indépendantes pour les mots de passe, les sessions, les keystores et le chiffrement des notes. Les clés EVM des trois comptes préchargés constituent une exception volontaire : ce sont des clés de test publiques, versionnées dans `.env.example`, utilisables uniquement sur cette chaîne isolée sans valeur. Une exploitation réelle devrait les remplacer.

7. Démarche de réalisation

La construction a suivi un ordre qui rendait chaque hypothèse testable.

1. **Formalisation.** Les quatre règles ont d'abord été exprimées en prédicats indépendants de l'interface.
2. **Contrat.** Le modèle minimal a été implémenté puis testé dans un EVM en mémoire, avant d'ajouter le réseau Besu.
3. **Réseau.** Le genesis et les quatre identités QBFT ont été générés par script. Les volumes séparés et le nombre de pairs ont ensuite été contrôlés.
4. **Passerelle.** L'application a été ajoutée une fois les appels du contrat stabilisés. Les lectures utilisent systématiquement l'adresse de l'utilisateur.
5. **Parcours réaliste.** L'inscription est passée d'une saisie d'adresse par le professeur à un cycle demande–approbation inscrit sur la chaîne.
6. **Durcissement.** Les protections web, le chiffrement des notes, HTTPS et les contrôles de production ont été ajoutés avant la recette cloud.
7. **Automatisation.** `make demo` reproduit l'installation ; `make verify` regroupe les tests de contrat, le parcours E2E et l'essai de consensus.

Plusieurs anomalies rencontrées ont conduit à des corrections structurantes :

Observation	Diagnostic	Correction
déploiement Solidity sans code	bytecode compilé avec <code>PUSH0</code> , absent du palier Berlin	compilation explicite pour <code>evm_version="berlin"</code>
erreur après évolution du contrat sur Azure	ancienne adresse réutilisée malgré une ABI incompatible	contrôle d'interface avant réutilisation, sinon redéploiement
CSS bloqué en HTTPS	URL générée en HTTP derrière Caddy	prise en compte de <code>X-Forwarded-Proto</code> , acceptable car l'API n'écoute que sur l'interface locale
suite du script SSH ignorée	<code>docker compose exec</code> consommait l'entrée standard	exécution avec <code>-T --interactive=false</code>
clés réseau impossibles à nettoyer	fichiers créés par Docker avec le propriétaire root	génération avec l'UID/GID de l'hôte

Ces problèmes n'ont pas été masqués par des contournements manuels : leurs causes ont été intégrées aux scripts afin qu'une installation neuve suive le même chemin que l'environnement de développement.

8. Validation

8.1 Tests automatisés

La suite `pytest` compte **23 tests**. Elle vérifie séparément les informations publiques, cours, TP, examen, correction, notes, rôles, inscription, refus, réinscription et historique append-only. Elle couvre aussi le rattachement obligatoire d'une correction à son examen et l'héritage de sa date, le chiffrement des notes, l'authentification du ciphertext, CSRF, les secrets de production et l'isolation des chemins de fichiers. Deux tests V4 vérifient la conservation des anciennes informations de classe et notes, ainsi que l'accès privé à ces dernières.

8.2 Parcours de bout en bout

Le smoke test agit par HTTP sur le réseau réel. Il se connecte comme enseignant, publie un cours et un examen, publie une note, puis contrôle les vues de deux étudiants. Il crée également un nouveau compte, envoie sa demande, vérifie la file d'attente et l'approbation. Enfin, il altère volontairement le fichier sur disque : le téléchargement est refusé parce que le SHA-256 ne correspond plus.

8.3 Réplication et quorum

Le test de conformité interroge chaque RPC depuis le réseau privé. Lors de la campagne Azure V4 du 8 septembre 2026, également réussie en local :

Essai	Résultat observé
état initial	4 validateurs, 3 pairs chacun et même hash au bloc comparé
arrêt d'un validateur	transaction finalisée par les 3 validateurs restants
arrêt d'un second validateur	transaction toujours en attente après le délai
restauration	rattrapage, retour à 3 pairs et même hash sur les quatre nœuds

Ce résultat démontre à la fois la cohérence des copies, le quorum et le rattrapage. Il ne se limite pas à vérifier qu'un conteneur répond.

L'artefact Solidity a en outre été recompilé avec la version fixée du compilateur. La comparaison de son SHA-256 avant et après une seconde compilation confirme que l'ABI et le bytecode versionnés sont reproductibles à partir du source testé.

9. Déploiement Azure

La démonstration cloud utilise une VM Ubuntu 22.04 `Standard_B2ms` (2 vCPU, 8 Gio), un disque Standard SSD de 30 Gio et la région France Central. Docker est installé par cloud-init ; le code est transféré par `rsync` sans `.env`, clés de nœuds ni données locales.

Caddy publie l'application sur un nom DNS Azure avec un certificat Let's Encrypt. Le pare-feu autorise 80 et 443 depuis Internet, et 22 seulement depuis l'adresse de l'administrateur. Les sondes externes ont confirmé que 8000, 8545 et 30303 sont fermés. HTTP renvoie une redirection permanente vers HTTPS ; CSP, HSTS et `X-Content-Type-Options` sont présents.

La recette finale du **8 septembre 2026** porte sur le **contrat V4** : les **23 tests**, le smoke test HTTPS et l'essai QBFT complet ont réussi sur la VM, comme en local. Un redémarrage réel de la VM a ensuite confirmé le retour automatique de l'application, du RPC et du contrat V4, grâce à la politique `restart: unless-stopped`. Besu nécessite un délai de démarrage : une réponse HTTP seule ne prouve pas encore la disponibilité de la chaîne.

Le script détecte un contrat incompatible et en déploie un nouveau, puis initialise les comptes de démonstration. Il ne migre pas les anciennes données : l'ancien contrat reste sur la chaîne, mais l'application utilise la nouvelle adresse. Les tests HTTP ajoutent des données fictives. Après la recette, la VM a été désallouée pour arrêter la facturation du calcul, sans supprimer les volumes persistants.

Le coût principal est le temps de calcul de la VM. La valeur observée dans l'API Azure Retail Prices pour une B2ms Linux en France Central était de 0,0944 USD/h au 5 septembre 2026, hors disque et IP [7]. Le script ajoute un arrêt automatique à 23 h UTC et une commande de désallocation. Il tente également de créer un budget mensuel de 10 unités avec alerte à 80 % lorsque le compte Azure possède les droits nécessaires. Une VM désallouée ne facture plus le compute, mais le disque et l'IP réservée restent facturables.

10. Limites et portée des résultats

La première limite concerne la **distribution**. Quatre processus, quatre clés, quatre volumes et un consensus réel sont présents, mais tous partagent une seule VM. Une architecture de production répartirait les validateurs entre plusieurs hôtes et plusieurs organisations. Le prototype démontre le mécanisme P2P, pas la résilience à la perte de la machine physique.

La deuxième limite concerne la **confiance**. Le consensus évite qu'un nœud unique décide de l'historique, mais la passerelle reste centrale pour l'identité, la conservation des fichiers et les clés étudiantes. L'enseignant est par ailleurs une autorité métier légitime pour les inscriptions et les publications. Le projet réduit donc le tiers de confiance pour la validation du registre sans le faire disparaître de toute l'application.

Enfin, le chiffrement protège les notes contre la lecture directe du stockage de la chaîne, mais la gateway détient la clé de déchiffrement. Les fichiers ne sont pas répliqués entre validateurs : leur intégrité est prouvée, pas leur disponibilité. Une évolution réelle demanderait des portefeuilles côté client, une gestion institutionnelle des identités, des validateurs répartis et un stockage de fichiers sauvegardé ou distribué.

Conclusion

ClassLedger répond au scénario demandé sans transformer la blockchain en simple argument d'interface. Les règles d'accès, l'historique des versions et les inscriptions sont exécutés par un contrat ; les quatre nœuds répliquent le même registre et QBFT impose un quorum mesuré en situation de panne. Le stockage hybride conserve une preuve immuable des fichiers sans placer leurs octets sur la chaîne.

La démarche a privilégié les preuves : chaque exigence possède un test, les propriétés P2P sont comparées entre les quatre nœuds, et le déploiement Azure a été recetté depuis Internet. Les limites — hôte physique unique, gateway custodiale, fichiers non répliqués — restent visibles. Le résultat est ainsi un prototype cohérent et reproductible, adapté à l'objectif pédagogique, sans prétendre être une plateforme scolaire de production.

Références

État de l'art arrêté au **5 septembre 2026** ; sources officielles et liens vérifiés le **8 septembre 2026**. Chaque référence correspond à un passage cité.

1. Microsoft Learn, [About Azure confidential ledger](#).
2. Microsoft, [Migrate Azure Blockchain Service](#), archive officielle : retrait du service le 10 septembre 2021 et alternatives autogérées sur VM.
3. Amazon Web Services, [Amazon Managed Blockchain pricing](#) : offres publiques, réseaux privés Hyperledger Fabric et postes de facturation.
4. IBM Documentation, [IBM Blockchain Platform 2.5.4 — product version no longer published](#).
5. Oracle, [Oracle Blockchain](#).
6. Hyperledger Besu, [Configure QBFT consensus](#).
7. Microsoft Azure, [Retail Prices API — requête Standard_B2ms](#), [France Central](#) et [documentation de l'API](#). Entrée retenue : `Virtual Machines BS Series`, compteur `B2ms`, 0,0944 USD/h, hors Windows ; la réponse de l'API évolue avec les tarifs.
8. Hyperledger Fabric, [Introduction](#).
9. FastAPI, [Templates](#) : rendu HTML avec Jinja2 et intégration des fichiers statiques.
10. web3.py, [Contracts](#) : ABI, déploiement, lectures `call`, transactions `build_transaction` et tests sur EVM en mémoire.